

Лекция №3

Циклы в JavaScript

Цель лекции

Научиться использовать циклы для многократного выполнения команд и обработки повторяющихся данных.

- использовать циклы for, while и do...while;
- применять счётчики и накопители;
- сочетать циклы с условиями;
- управлять выполнением цикла с помощью break и continue.

1. Что такое цикл

Цикл — это конструкция, позволяющая многократно выполнять один и тот же блок команд. Основные циклы JavaScript: for, while и do...while.

2. Цикл for

for удобно использовать, когда заранее известно количество повторений. Общий вид: for (начало; условие; изменение) { команды }.

```
for (let i = 1; i <= 5; i++) {  
    console.log(i);  
}
```

3. Как работает for

В выражении let i = 1 создаётся счётчик и задаётся его начальное значение. Условие i <= 5 определяет, пока цикл выполняется. Команда i++ увеличивает счётчик после каждого повторения.

```
for (let i = 1; i <= 3; i++) {  
    console.log(i);  
}
```

4. Изменение счётчика

Счётчик можно изменять не только на единицу. Например, i += 2 увеличивает его на 2, а i-- уменьшает на 1.

```
for (let i = 2; i <= 10; i += 2) {  
    console.log(i);  
}  
  
for (let i = 5; i >= 1; i--) {  
    console.log(i);  
}
```

5. Вычисления в цикле

Для накопления результата используется переменная-накопитель. Например, найдём сумму чисел от 1 до 5.

```
let sum = 0;  
  
for (let i = 1; i <= 5; i++) {  
    sum += i;  
}  
  
console.log(`Сумма: ${sum}`);
```

6. Подсчёт количества

С помощью счётчика можно определить количество значений, удовлетворяющих условию. Здесь цикл объединяется с if.

```
let count = 0;  
  
for (let i = 1; i <= 10; i++) {  
    if (i > 5) {  
        count++;  
    }  
}  
  
console.log(`Количество: ${count}`);
```

7. Цикл while

while выполняет команды, пока условие истинно. Он удобен, когда количество повторений заранее неизвестно.

```
let i = 1;

while (i <= 5) {
  console.log(i);
  i++;
}
```

8. Бесконечный цикл

В while необходимо изменять переменную, участвующую в условии. Если условие всегда остаётся истинным, возникает бесконечный цикл.

```
let i = 1;

while (i <= 5) {
  console.log(i);
  i++;
}
```

9. Цикл do...while

В do...while условие проверяется после выполнения тела цикла, поэтому команды выполнятся как минимум один раз.

```
let i = 1;

do {
  console.log(i);
  i++;
} while (i <= 5);
```

10. Сравнение циклов

for применяют, когда известно количество повторений; while — когда нужно повторять действия, пока выполняется условие; do...while — когда действие должно выполниться хотя бы один раз.

11. Оператор break

break полностью завершает цикл раньше обычного окончания.

```
for (let i = 1; i <= 10; i++) {
  if (i === 5) {
    break;
  }
  console.log(i);
}
```

12. Оператор continue

continue пропускает текущее повторение и переходит к следующему.

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) {
    continue;
  }
  console.log(i);
}
```

13. Цикл и условие

Циклы часто используются совместно с if. Это позволяет выполнять действия только для значений, удовлетворяющих условию.

```
for (let stock = 0; stock <= 5; stock++) {
  if (stock > 0) {
    console.log(`Остаток: ${stock}`);
  }
}
```

14. Вложенные циклы

Один цикл можно разместить внутри другого. Для каждого повторения внешнего цикла внутренний цикл выполняется полностью.

```
for (let i = 1; i <= 3; i++) {  
  for (let j = 1; j <= 2; j++) {  
    console.log(`i = ${i}, j = ${j}`);  
  }  
}
```

15. Комплексный пример

Интернет-магазин продаёт товар стоимостью 1500 рублей. Выведем стоимость покупки от одной до пяти единиц.

```
const price = 1500;  
  
for (let count = 1; count <= 5; count++) {  
  const total = price * count;  
  console.log(`${count} шт. — ${total} руб.`);  
}
```

16. Типичные ошибки

Частые ошибки: неверное условие начала/окончания цикла; отсутствие изменения счётчика в while; создание бесконечного цикла; путаница между break и continue.

Краткие выводы

Циклы позволяют многократно выполнять блок команд. Основные конструкции: for, while и do...while. Для досрочного завершения используется break, для пропуска текущего повторения — continue. Циклы особенно важны при дальнейшей обработке массивов товаров, пользователей, заказов и результатов запросов к базе данных.